

Low-Rate Quantization in Hybrid Language Models: Codebook Dimension, Error Compensation, and Coverage

Working research draft

September 22, 2026

Status: working draft. Not submission ready.

No quantum-hardware, quantum-advantage, or quantization-aware-training result is reported. No fully quantized model, runtime memory measurement, or latency measurement is reported.

Abstract

We report a controlled study of post-training weight quantization below two bits in a hybrid linear-attention language model, asking which factors move validation loss under measured storage budgets: codebook dimension, error compensation, index rate, and which tensors are converted at all. On the DeltaNet QKV projections of a 0.8B model a shared eight-dimensional codebook reduces damage relative to a learned three-level scalar code by 0.036118 NLL (paired block bootstrap [0.033768, 0.038414]) at 0.43% more payload bytes, and the direction reproduces on the 48 corresponding projections of a 27B checkpoint over its complete validation stream (0.007121, [0.005725, 0.008513], 0.019% more bytes). The advantage keeps its sign under eight refits of the quantizer, on held-out text, on a second domain, and at contexts to 2048 tokens; its magnitude varies across those conditions (0.031 to 0.038 across refits), and we quote ranges rather than single margins because the spread across refits is of the same order as the text-bootstrap interval. Two factors dominate the codec choice. GPTQ-style error compensation removes 68–69% of the damage on both tensor families, roughly three times what the codec contributes, while *raising* plain weight MSE — inverting a proxy that earlier work, including our own earlier drafts, relied on. Index rate dominates in turn: from 1.6 to 4.0 bits damage falls approximately exponentially, halving about every 0.47 bits, with no knee across a $2.4\times$ range of stored bytes. Coverage matters more again — MLP weights are $2.79\times$ more damaging per parameter than the projections — yet a Hessian-derived per-tensor ranking built to exploit that performs *worse than random selection* of the same byte budget, the second forward-side ranking failure we record. These are partial-coverage, payload-storage results on two checkpoints and two corpora; they establish neither full-model quality nor runtime feasibility, and we do not claim a new algorithm.

1 Scope and contributions

Vector quantization with Hessian-aware compensation, and Hadamard incoherence processing with eight-dimensional codes, are established ideas [1, 2, 3, 4]. We do not claim a new algorithm; the block extension of GPTQ in Section 4 is a routine generalization and we treat it as implementation, not contribution. What we offer is controlled evidence, with explicit boundaries, about a setting the literature covers thinly: rates below two bits, on the recurrent projections of a hybrid model, with storage measured from bytes that are actually written and weights that are actually read back.

Concretely:

1. **Dimension.** A shared 6561-point eight-dimensional codebook beats a learned three-level scalar code at near-equal payload bytes, on two tensor families of a 0.8B model and on the QKV projections of a 27B model (Table 2). Eight dimensions earn their place: on the frozen 27B run they beat four by 0.004034 (CI [0.002715, 0.005334]), more than half of the eight-dimensional code’s entire advantage over scalar. A 512-block pilot had suggested the opposite, with four dimensions capturing 95.4% of the gain; that estimate did not survive the full stream.
2. **Compensation dominates the codec.** Error compensation is worth 68–69% of the damage against the codec’s 6–32%, and it improves loss while degrading weight MSE.
3. **Rate dominates in turn,** falling exponentially with no knee over the measured interval (Table 6).
4. **Coverage dominates all of them,** and the obvious way to exploit that — ranking tensors by activation-weighted error per byte — is worse than random.
5. **Two negative results on proxies and one on method.** Weight MSE and activation-weighted error each predict loss inside the regime where they are fitted and fail outside it; a self-consistent calibration loop is unnecessary because the dependency graph is acyclic.
6. **Robustness of the main result.** On these 0.8B DeltaNet projections the eight-dimensional code’s advantage survives refitting over calibration draws and seeds, held-out text, a second domain, and context extension through 2048 tokens. Its magnitude varies across those conditions, and finer scalar grouping shows no consistent benefit in any of them.

2 What this paper does not establish

We separate these from the results because earlier drafts of this work did not, and the reader should not have to reconstruct the boundary.

- **No full-model conversion.** Every measurement covers a subset: 15.1% of parameters (0.8B QKV), 35.1% (0.8B MLP), 50.2% (both), or 9.36% (27B QKV). No end-to-end quantized checkpoint was produced or evaluated.
- **No runtime claim.** We report payload bytes and complete artifact file bytes. Packed weights fitting in VRAM does not establish that execution fits: KV cache, recurrent state, activations, kernel workspaces and decoding buffers are unmeasured. There is no packed inference kernel.
- **No four-bit deployment claim.** A historical +0.008393 NLL four-bit measurement exists, but it is a 0.8B projection-subset result (0.24% of baseline NLL, 0.84% relative perplexity — distinct quantities). It is not a 27B result and not a full-coverage result. An earlier draft used it as both; that claim is withdrawn.
- **No causal scaling statement.** The two checkpoints differ in generation, coverage, tensor dimensions, calibration size and evaluated text extent. That the 27B margins are smaller is an observation, not an explained consequence of scale.
- **No equivalence results.** Where an interval includes zero we say *inconclusive*. We prespecified no equivalence margin, and failing to reject zero is not evidence of no effect.

Run	target params	share	eval targets	calib. tokens
ternary_task_v4	0	0.0%	261,284	65,536
mlp_task_v1	264,241,152	35.1%	261,284	–
mixed_alloc_v1	377,487,360	50.2%	261,284	–
rate_curve_v2	377,487,360	50.2%	261,284	–
qwen38_confirm_v1	2,516,582,400	9.4%	65,536	–
qwen38_confirm_v2	2,516,582,400	9.4%	261,284	65,536

Table 1: Protocol and coverage for every run reported. Coverage and calibration differ between checkpoints, so cross-run differences are not attributable to scale alone.

3 Protocol

All arms share one protocol, and the properties below are the ones earlier versions of this work got wrong.

Storage is measured, not derived. Every codec serializes to packed bytes — five trits per byte, or c base- K codes per `uint64` with $K^c < 2^{64}$ — and bits per weight is computed from `len(bytes)`, not from $\log_2 K/d$. Multi-symbol packing makes the ternary and vector formats cost the same per index to within a few bytes per tensor, which is the conservative choice as well as the realistic one.

The evaluated weights are the stored weights. Group scales are rounded to FP16 *before* quantization, and every tensor is decoded back from its serialized bytes and required to match the quantizer’s reconstruction exactly before it is installed. In the confirmation run the artifact is additionally written to disk, reloaded and re-decoded. Omitting this check cost an earlier draft six points of headline: FP16 scale storage costs a 6561-point codebook +0.019 NLL while leaving a 3-point code unchanged.

Byte matching is near-equal, not literal. Where we say byte-matched, dimension-4 costs +0.0029% over scalar at 0.8B and +0.00012% at 27B; dimension-8 costs +0.43% and +0.0193%. Tables report five decimals.

Evaluation. Token-weighted NLL over the complete validation stream including the ragged final block, with a paired block bootstrap over ≥ 5000 resamples. Calibration uses the training split. Table 1 gives coverage and calibration size per run; they are not identical across runs, which is why cross-run differences are not attributed to scale.

4 Method

Rotation is a blockwise randomized Hadamard transform on the input dimension. Codebooks are fitted by Lloyd/ k -means in group-normalized space and shared across all target tensors, charged once.

Extending GPTQ to a codec that quantizes d columns jointly is the only non-standard step. For a block of columns J the cost charged by the GPTQ derivation and the compensation applied to the remainder are

$$\|(W_J - Q_J) U_{JJ}^{-1}\|_F^2, \quad W_{\text{rest}} \leftarrow (W_J - Q_J) U_{JJ}^{-1} U_{J,\text{rest}},$$

with U the upper Cholesky factor of H^{-1} . The within-block decision is therefore a metric nearest-neighbour search with $M = U_{JJ}^{-1}$: transform points and codebook by M and search normally. At $d = 1$ this must reduce to textbook scalar GPTQ, and our test suite verifies it does so bit-exactly against an independently written reference. We regard this as a routine generalization.

Contrast	Run	Δ NLL	95% CI	byte ratio
VQ8 vs scalar (0.8B)	0.8B/GPTQ	-0.036118	$[-0.038414, -0.033768]$	1.00430
VQ8 vs scalar (27B)	27B/frozen	-0.007121	$[-0.008513, -0.005725]$	1.00019
VQ4 vs scalar (0.8B)	0.8B/GPTQ	-0.024667	$[-0.027041, -0.022348]$	1.00003
VQ4 vs scalar (27B)	27B/frozen	-0.003087	$[-0.004364, -0.001807]$	1.00000
VQ8 vs scalar g64 (0.8B)	0.8B/GPTQ	-0.034303	$[-0.036660, -0.031971]$	0.93644
VQ8 vs scalar g64 (27B)	27B/frozen	-0.007836	$[-0.009247, -0.006356]$	0.93261
VQ8 vs VQ4 (27B)	27B/frozen	-0.004034	$[-0.005334, -0.002715]$	1.00019

Table 2: Primary and secondary contrasts, matched recipe across checkpoints. The 0.8B rows come from the GPTQ run, not the earlier uncompensated one. Each row names the run it comes from; rows from different runs are not matched comparisons. Byte ratios are printed to five decimals because “byte-matched” here means near-equal payload, not literal parity.

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
scalar3_rot	1.7250	24.42	+0.356599	$[+0.352028, +0.361286]$
scalar3_rot_actw	1.7250	24.42	+0.361084	$[+0.356485, +0.365764]$
scalar3_rot_gptq	1.7250	24.42	+0.113179	$[+0.109919, +0.116351]$
vq8_rot	1.7324	24.52	+0.240180	$[+0.236367, +0.243903]$
vq8_rot_actw	1.7324	24.52	+0.227827	$[+0.224049, +0.231700]$
vq8_rot_gptq	1.7324	24.52	+0.077061	$[+0.074362, +0.079868]$
vq8_rot_gptq_actwfit	1.7324	24.52	+0.077267	$[+0.074642, +0.079910]$
vq4_rot_gptq	1.7251	24.42	+0.088512	$[+0.085730, +0.091370]$
scalar3_g64_rot_gptq	1.8500	26.19	+0.111363	$[+0.108285, +0.114369]$
vq8_rot_gptq_seq	1.7324	24.52	+0.075171	$[+0.072454, +0.077936]$

Table 3: Codec and recipe ablation on the 0.8B DeltaNet QKV projections. Source: `results/ternary_task_v4` (0.8B, GPTQ recipe); BF16 NLL 3.436710.

5 Results

5.1 Dimension helps, and eight dimensions earn their place

At near-equal payload bytes the eight-dimensional code beats the learned scalar ternary code on both 0.8B tensor families and on the 27B projections (Table 2). Against a scalar control that spends 6.4% *more* bytes the margin is -0.034303 $[-0.036660, -0.031971]$ at 0.8B under the matched compensated recipe; at 27B, over the complete validation stream, it is -0.007836 $[-0.009247, -0.006356]$ at a payload ratio of 0.93261 — the vector code wins while storing 6.7% *fewer* bytes. A 512-block pilot of the same comparison was inconclusive (-0.001736 $[-0.004444, +0.000923]$).

On the frozen confirmation, dimension-8 beats dimension-4 by -0.004034 $[-0.005334, -0.002715]$ — more than half of dimension-8’s entire advantage over the scalar code. The 512-block pilot had dimension-4 capturing 95.4% of the gain, with dimension-8 ahead by only 0.000181; that reading did not survive the complete stream. The difference is not sampling: the pilot’s blocks and the remaining 1,530 agree with each other, and BF16 loss on the pilot’s blocks is identical across runs to all printed digits. What differs is calibration size (32,768 against 65,536 tokens), which moved the storage-dominance contrast across non-overlapping intervals on identical evaluation data. Every interval reported here is conditional on one fitted quantizer, and calibration variability is not inside it.

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
mlp_scalar3_rot	1.7250	56.98	+2.119720	[+2.104286, +2.134513]
mlp_scalar3_rot_gptq	1.7250	56.98	+0.658007	[+0.648510, +0.667584]
mlp_vq4_rot_gptq	1.7250	56.98	+0.552558	[+0.544087, +0.561040]
mlp_vq8_rot_gptq	1.7282	57.08	+0.500777	[+0.492819, +0.508608]
mlp_vq8_rot_gptq_pertype	1.7345	57.29	+0.521963	[+0.514011, +0.529926]
mlp_scalar3_g64_rot_gptq	1.8500	61.11	+0.656502	[+0.647276, +0.665713]

Table 4: The same recipe on all 72 MLP tensors of the 0.8B model. Source: `results/mlp_task_v1` (0.8B MLP, GPTQ recipe); BF16 NLL 3.436710.

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
scalar3_rot_gptq	1.7250	542.64	+0.010958	[+0.009366, +0.012561]
vq4_rot_gptq	1.7250	542.64	+0.007871	[+0.006436, +0.009331]
vq8_rot_gptq	1.7253	542.74	+0.003837	[+0.002420, +0.005250]
scalar3_g64_rot_gptq	1.8500	581.96	+0.011673	[+0.010078, +0.013264]

Table 5: Scale transfer, frozen confirmation: 48 DeltaNet QKV projections of Qwen3.8-27B over the complete validation stream. Source: `results/qwen38_confirm_v2` (27B, GPTQ recipe, frozen full stream); BF16 NLL 2.691117.

5.2 Compensation dominates the codec, and inverts the proxy

Error compensation removes 68.3% of the damage on the 0.8B projections and 69.0% on the MLPs — both far larger than the codec effect. It achieves this while *raising* plain weight MSE by 38% ($7.18 \rightarrow 9.89 \times 10^{-5}$), so across recipes weight MSE is anti-predictive even though within a fixed recipe its ordering matches the loss ordering exactly. Diagonal activation weighting, by contrast, is worthless: harmful on scalar codes (+0.004485), and a tie once compensation is present (+0.000206), because the block metric already carries the diagonal and the cross terms.

5.3 Rate dominates compensation

Over 1.600 to 4.000 index bits at fixed 50.2% coverage, damage halves every 0.451, 0.506, 0.416, 0.530 and 0.468 bits — approximately exponential across a $2.40\times$ range of payload bytes, with no knee visible (Table 6). Six rates on one checkpoint at one coverage support “approximately exponential over this interval” and nothing stronger.

The four-bit point is measured on these same tensors: +0.018572 NLL, +1.87% perplexity. It is *not* interchangeable with the historical four-bit figure of +0.008393, which covers 15.1% of parameters rather than 50.2%.

5.4 Coverage dominates rate, and cannot be exploited by ranking

MLP weights are $2.79\times$ more damaging per parameter than the projections. Given a fixed extra-byte budget, confining it to the MLP family beats spending it on the projections by -0.061105 [$-0.064430, -0.057804$].

The natural refinement — rank all 90 tensors by activation-weighted error removed per extra byte — *fails*. It promotes 14 of 18 projections and none of the 48 up/down projections, and loses to random selection of the same budget by +0.061244 [+0.058083, +0.064344]. Per parameter the projections carry $5.7\times$ more activation-weighted error while causing $2.79\times$ less loss damage, so

K	index bpw	total bpw	payload MB	Δ NLL	95% CI
81	1.600	1.7250	81.40	+0.637829	[+0.628446, +0.647323]
84	1.600	1.7250	81.40	+0.629249	[+0.619764, +0.638577]
255	2.000	2.1250	100.27	+0.340476	[+0.334386, +0.346756]
565	2.286	2.4108	113.76	+0.230274	[+0.225334, +0.235236]
1625	2.667	2.7920	131.74	+0.121974	[+0.118594, +0.125236]
7131	3.200	3.3262	156.95	+0.060716	[+0.058676, +0.062752]
65535	4.000	4.1361	195.17	+0.018572	[+0.017476, +0.019687]

Table 6: Uniform rate sweep at fixed coverage (50.2% of parameters), dimension 4 throughout so rate varies alone; six rates from 1.600 to 4.000 index bits. Stored-byte range $2.40\times$. Source: `results/rate_curve_v2`; BF16 NLL 3.436710.

Contrast	bootstrap half-width	range: draws	seeds	size*
VQ8 vs scalar	0.002416	0.004255	0.005057	0.004474
VQ8 vs scalar g64	0.002416	0.007078	0.005057	0.007110
VQ8 vs VQ4	0.002324	0.003893	0.006350	0.011189
g64 vs g128	0.001983	0.005548	0.000000	0.007441

Table 7: Conditional text-bootstrap half-width beside the range of each contrast over refits. These are different statistics and are not divided: a range over a few point estimates has no coverage interpretation. *Calibration size changes the experimental condition rather than resampling it. The zero for seed on the scalar-only contrast is a harness check: both arms use a deterministic Lloyd fit, so a seed cannot move it. Source: `results/calib_variability_v1`.

across families the proxy is anti-correlated with its target. The quantity measures error injected at a layer’s output and carries no information about propagation to the loss. This is the second forward-side ranking failure we record; a Fisher-based diagnostic gave Spearman $+0.015$.

5.5 How far the main result travels

Two studies probe the conditions under which the eight-dimensional advantage was measured, one varying the fit and the other the evaluation.

Refitting (Table 7). Eight refits over three disjoint calibration draws, three codebook seeds and four calibration sizes. All three primary contrasts keep their sign in 8/8 refits; VQ8 against scalar ranges from -0.037809 to -0.031061 . The ranges over refits are of the same order as, and in several cases larger than, the text-bootstrap half-width, so magnitudes are quoted as ranges rather than single margins. VQ8’s advantage grows monotonically with calibration size (-0.033335 at 16k tokens to -0.037809 at 131k), which is a condition effect rather than noise.

Varying the evaluation (Table 8). One fitted quantizer evaluated on the never-used wikitext-2 test split, on TinyStories, and at 512 and 2048 token contexts. All three primary contrasts exclude zero in every configuration. Through 2048 tokens there is no ordering reversal and no large increase in aggregate damage, though VQ8’s advantage does shrink about 18% from the 128-token setting; aggregate loss averages over positions, so this is not evidence against recurrent-state drift. Out of domain, VQ8’s absolute advantage grows ($0.038806 \rightarrow 0.043775$) while the fraction of scalar damage it removes falls ($32.0\% \rightarrow 24.0\%$); we do not attribute the higher out-of-domain damage to calibration mismatch, which would need two calibration domains crossed with two evaluation domains.

Contrast	wt2_val_128	wt2_test_128	tiny_128	wt2_test_512	wt2_test_2048
VQ8 vs scalar	-0.036118	-0.038806	-0.043775	-0.035788	-0.031832
VQ8 vs scalar g64	-0.034303	-0.039543	-0.054717	-0.035518	-0.031779
VQ8 vs VQ4	-0.011451	-0.011840	-0.015677	-0.015220	-0.012270
g64 vs g128	-0.001815 [†]	+0.000737 [†]	+0.010942	-0.000270 [†]	-0.000053 [†]

Table 8: One fitted quantizer; only the evaluation varies. These are two corpora and three context lengths, not five independent replications — the three `wt2_test` columns share their text and differ only in blocking. [†] marks an interval including zero. Source: `results/generalization_v1`.

The scalar group-size control has no stable sign: it flips across refits and, on TinyStories, excludes zero in the *unfavourable* direction (+0.010942). Our leading hypothesis is that moving from g128 to g64 does not enlarge a solution space containing the g128 solution — group scales are derived from group maxima rather than chosen, and the shared codebook is refit in the new normalized space — so extra scale bytes guarantee neither lower reconstruction error nor lower loss. That is a hypothesis; a direct test would hold the codebook fixed across group sizes.

6 Negative results retained from earlier work

Two hypotheses were pursued at length and refuted; they are retained because their failure mode recurs.

Polar weight codes. Storing a magnitude and a phase index per weight pair appeared to beat an equal-byte real four-bit code. It beat only a control with a *uniform* codebook; giving the scalar control the same Lloyd–Max fitting reverses the ranking. Repairing the polar assignment changes MSE by 0.035%, because with five phase bits $1 - \cos(\pi/32) \leq 4.8 \times 10^{-3}$ while one radius-grid step is $\approx 14\%$ of the row scale.

Self-consistent calibration. Solving activation-aware calibration as a DFT-style fixed point is unnecessary: the forward-only dependency graph is acyclic, so one ordered pass is exact, and closed-form one-pass corrections already exist [6]. Measured, sequential capture is worth -0.001890 $[-0.003421, -0.000315]$ at $3.5\times$ the cost.

The pattern. Both, and the ranking failure above, are the same error: a method beat a control that had been given less tuning, or a proxy was trusted outside the regime where it was fitted. Every correction in this paper came from a control — a fitted scalar baseline, a random allocation, a bit-exactness assertion — and not from inspection.

7 Reproducibility and limitations

Runs record a manifest: arguments, source hashes, checkpoint hashes, library and CUDA versions, device map, GPU budget, calibration size, and the evaluated versus available target counts. Tables in this paper are generated directly from saved results by `experiments/make_tables_v2.py`; an earlier draft mixed an uncompensated margin into a compensated comparison, and generation is how we prevent recurrence rather than by care.

An exactness failure, observed once and not reproduced. One run at a 17 GiB GPU weight budget raised the per-tensor bit-exactness check with a decoded-versus-reference difference of 0.036. Seven subsequent runs at that budget, across two script versions in fresh processes, instead terminated with an identical out-of-memory error before reaching that state; 15 and 16 GiB are exact. The corruption was never reproduced and no failing tensor was captured, so it could not be

localized. One observation of a wrong value stands against seven of a clean failure, and an earlier draft’s framing of this as a characterized budget-sensitive defect is withdrawn. Every reported run used 15 GiB, every tensor passed the strict check, and the confirmation run additionally re-decodes each artifact from disk.

Statistical scope. Intervals measure text-sampling variability conditional on one fitted quantizer, and that conditioning is load-bearing. Refitting over three calibration draws and three codebook seeds produces ranges of the same order as, and in several cases larger than, the text-bootstrap half-width; calibration size is reported separately again, as a condition change rather than a further sample. We report the two statistics side by side and do not divide them: a range over a handful of refits and a bootstrap half-width are different quantities, and their ratio has no coverage interpretation. Directions survive — the vector code wins in all eight refits on all three primary contrasts — but a single six-decimal margin from one refit is more precise than the procedure warrants, so magnitudes are quoted as ranges (-0.031 to -0.038 for VQ8 against scalar). The scalar group-size contrast changes sign across refits and across evaluation domains. The 27B intervals carry the same conditioning, with refit variability unmeasured there.

Generalization scope. The five evaluation configurations in Section 5 are two corpora and three context lengths, not five independent replications; the three wikitext-test rows share their text. No ordering reversal and no large increase in aggregate damage was observed through 2048 tokens, which is not evidence of absence of recurrent-state drift — aggregate loss averages over positions, and position-resolved loss was not measured. Higher damage out of domain is reported without attributing it to calibration mismatch; isolating that needs two calibration domains crossed with two evaluation domains, which was not run.

References

- [1] GPTVQ: The Blessing of Dimensionality for LLM Quantization. <https://arxiv.org/abs/2402.15319>
- [2] QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. <https://arxiv.org/abs/2402.04396>
- [3] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. <https://arxiv.org/abs/2210.17323>
- [4] QuIP: 2-Bit Quantization of Large Language Models With Guarantees. <https://arxiv.org/abs/2307.13304>
- [5] Extreme Compression of Large Language Models via Additive Quantization. <https://arxiv.org/abs/2401.06118>
- [6] GPTAQ: Efficient Finetuning-Free Quantization for Asymmetric Calibration. <https://arxiv.org/abs/2504.02692>